

Training and Prediction Report (BioPoints)

Date: 2026/05/24

Team: Samuel Cesar and Marcelo Silva

Summary

Training and Prediction Report (BioPoints)	1
1. Executive Summary	2
2. Problem Understanding and Hypotheses	3
2.1. Business Context	3
2.2. Initial Hypotheses	3
3. Data Engineering and Features	4
3.1. Data Sources	4
CO2 Emission by Vehicles (Open Data/Historical Base): A tabular dataset containing historical reference data for thousands of vehicle models. It maps categories (Make, Model, Vehicle Class, Transmission, Fuel Type) and technical specifications (Engine Size, Cylinders, City/Highway/Combined Fuel Consumption). This serves both as a Machine Learning model dataset and for reference queries (VehicleSpec table)	4
3.2. Processing and Feature Engineering	4
4. Modeling and Scientific Validation	5
4.1. Architecture and Algorithms	5
4.2. Validation Strategy	6
4.3. Performance and Metrics	7
5. Model Interpretability (XAI)	9

1. Executive Summary

- **The Problem:** The CO₂ prediction model needs to estimate the emissions that a given vehicle would produce per kilometer using fossil fuels.
- **The Scientific Solution:** Create a regression model trained with data from the co2.csv dataset, with the objective of returning the CO₂ emission value based on the vehicle data provided by the user.
- **Projected Impact:**
 - Provide a predictive emissions model that estimates CO₂ savings per fueling event.

2. Problem Understanding and Hypotheses

2.1. Business Context

The project operates at the intersection of Smart Cities, Climate Change, and Energy Transition, using Salvador, Bahia, as a pilot environment. The primary business objective is to accelerate the transition from fossil fuels to biofuels (specifically biodiesel) through a measurable and financially rewarding incentive structure.

2.2. Initial Hypotheses

- **Hypothesis 1:** A vehicle's CO₂ emissions from fossil fuels can be accurately predicted using specific technical parameters (Engine Size, Cylinders, Transmission, Fuel Type, and Combined Fuel Consumption).
 - **Status: Confirmed.** The system uses a regression model trained on the 'co2.csv' dataset to map these input features directly to the target variable (CO₂_Emissions_g_per_km), establishing a reliable basis for calculating savings.
- **Hypothesis 2:** A reward system based solely on total fuel volume is insufficient; it must incorporate the vehicle's inherent efficiency to incentivize sustainable behavior.
 - **Status: Confirmed.** Business logic validates this by integrating an efficiency_multiplier (derived from L/100 km metrics) alongside CO₂ savings, ensuring drivers of more efficient vehicles receive proportional bonuses.

3. Data Engineering and Features

3.1. Data Sources

CO2 Emission by Vehicles (Open Data/Historical Base): A tabular dataset containing historical reference data for thousands of vehicle models. It maps categories (Make, Model, Vehicle Class, Transmission, Fuel Type) and technical specifications (Engine Size, Cylinders, City/Highway/Combined Fuel Consumption). This serves both as a dataset for the Machine Learning model and for reference queries (VehicleSpec table).

3.2. Processing and Feature Engineering

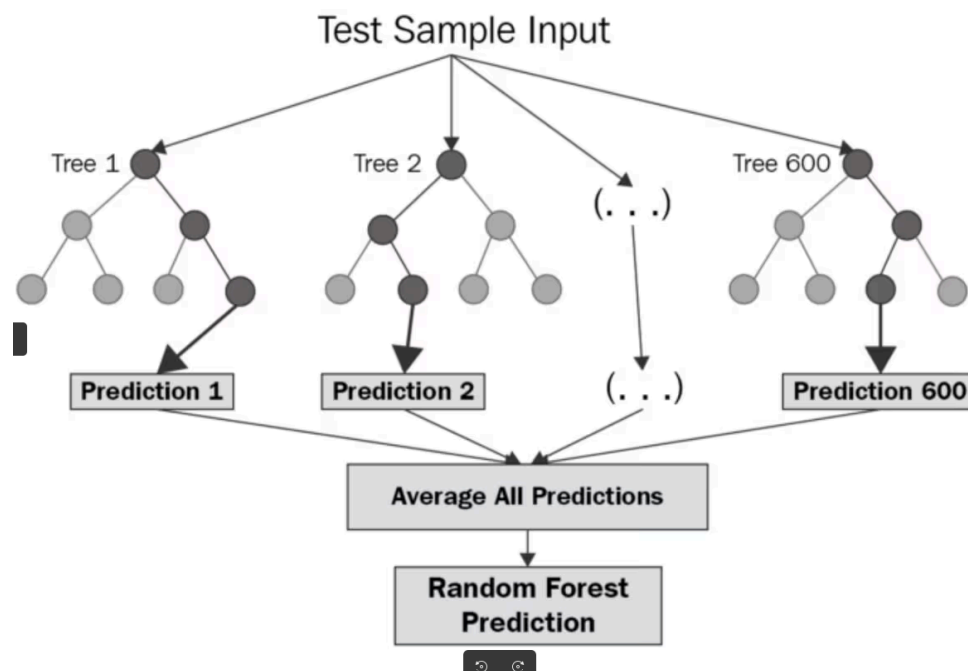
- **Processing:** Column headers were mapped to robust naming conventions in programming terms (e.g., Vehicle Class to Vehicle_type, CO2 Emissions(g/km) to CO2_Emissions_g_per_km) to avoid parsing errors in the API and pipeline.
- **Categorical Encoding:** Due to high cardinality and the string nature of specific columns, a ColumnTransformer with OneHotEncoder(handle_unknown='ignore') was implemented directly in the scikit-learn pipeline for Transmission, Fuel_Type, and Vehicle_type. This prevented errors during inference if a newly registered vehicle possessed a version or class never seen before.

4. Modeling and Scientific Validation

4.1. Architecture and Algorithms

Algorithm: RandomForestRegressor (n_estimators=100, random_state=42)¹.

Justification: Initially, a literature review was conducted based on the book *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*, especially the sections on Bagging (p. 190), Boosting (p. 196), and Random Forests (p. 193). Since I was dealing with a dataset mostly composed of numerical car features, it would be interesting to use a method that generated multiple predictions, finding an average of values, rather than sequential approaches like boosting, where each tree seeks to correct the errors of the previous ones. That said, Random Forest was chosen due to its high resilience in handling tabular data "mixed" between numerical and one-hot encoded text. It captures non-linear relationships (e.g., the interaction between 8 cylinders and how a specific transmission impacts emissions exponentially rather than linearly) and is robust to outliers² compared to standard linear regression models.



1: n_estimators: Number of trees; random_state=value to generate shuffling in the data output
(Source:

<https://medium.com/data-science-collective/what-the-heck-is-random-state-24a7a8389f3d>)

2: Values outside the standard of the rest of the data.

Architecture: The model is encapsulated in a .pkl object containing the complete preprocessing pipeline. When a fueling event hits the NestJS backend, the backend dynamically initiates a Python subprocess to load the model, inject the JSON payload of vehicle specs, and return the co2_fossil_predicted_g_km to calculate points.

4.2. Validation Strategy

Hold-out Strategy and Data Leakage Prevention: Validation was performed using a 20% hold-out set (X_test, y_test). Fundamentally, data preprocessing (OneHotEncoder) was encapsulated within the sklearn.pipeline.Pipeline. This ensures that data transformations are fitted to the training set during pipeline.fit(), preventing any statistical leakage from the test set into the model parameters before evaluation.

```
[13]: from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import OneHotEncoder
      from sklearn.compose import ColumnTransformer
      from sklearn.pipeline import Pipeline
      from sklearn.ensemble import RandomForestRegressor

      cat_features = ['Transmission', 'Fuel_Type', 'Vehicle_type']
      num_features = ['Engine_Size_L', 'Cylinders', 'Fuel_Consumption_Comb_L_per_100_km']

      preprocessor = ColumnTransformer(
          transformers = [
              ('num', 'passthrough', num_features),
              ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
          ])
      X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

[14]: X_train

```
[14]:
```

	Engine_Size_L	Cylinders	Transmission	Fuel_Type	Fuel_Consumption_Comb_L_per_100_km	Vehicle_type
6590	3.0	6	AS8	Z	9.9	SUBCOMPACT
6274	4.0	6	A5	X	12.7	PICKUP TRUCK - SMALL
2251	3.0	6	M6	Z	11.7	COMPACT
3149	3.4	6	AM7	Z	9.8	TWO-SEATER
4362	2.0	4	AS8	Z	8.7	STATION WAGON - MID-SIZE
...	...	...	...	...	...	...
5191	2.0	4	M6	Z	9.0	SUBCOMPACT
5226	3.5	6	AV7	X	9.1	MID-SIZE
5390	3.5	6	AS8	X	10.4	SUV - STANDARD
860	2.5	4	AV	X	8.6	SUV - SMALL
7270	4.0	8	AM8	Z	11.9	FULL-SIZE

5908 rows × 6 columns

4.3. Performance and Metrics

Evaluation Metric	Achieved Result	Business Interpretation
R² Score	0.996429065879812	Represents the percentage of variation in the vehicle's CO ₂ emissions. A high R ² proves that the chosen technical characteristics are determinants of actual emissions, ensuring a fair baseline for each driver.
MAE (Mean Absolute Error)³	1.9034969967008641	Translates the error margin into tangible operational terms. An extremely low MAE means that the co2_fossil_predicted_g_km value generated per fueling event is solid, ensuring that BioPoints are distributed equitably, without financial detriment to the platform.

$$MAE = \frac{1}{n} \sum \left| y - \hat{y} \right|$$

Diagram annotations:

- $\frac{1}{n}$: Divide by the total number of data points
- y : Actual output value
- $\hat{y}$: Predicted output value
- $\sum$: Sum of
- $|y - \hat{y}|$: The absolute value of the residual

3:

Note: In the test, I also use an example with a dataset without the 'Fuel_Consumption_Comb_L_per_100_km' variable, forcing the model to use other vehicle characteristics to generate the CO2 gas emission value, thus avoiding any possible Data Leakage. This test results in an R^2 Score of 0.9303714085442931 and an MAE of 11.061718682781224 g/km.

```
R2 Score: 0.9303714085442931
```

```
MAE: 11.061718682781224 g/km
```

```
['../ml/models/co2_model_without_fuel_consumption.pkl']
```


5. Model Interpretability (XAI)

Main Decision Factors:

- **Fuel_Consumption_Comb_L_per_100_km:** Demonstrates the strongest direct and positive correlation with CO₂ emissions. This technical validation mathematically proves the project's core business rule: fuel consumption directly translates into emissions, which is why the efficiency_multiplier is so crucial for reward logic.

```
[4]: import seaborn as sns
import matplotlib.pyplot as plt

direct_correlation = df['CO2_Emissions_g_per_km'].corr(df['Fuel_Consumption_Comb_L_per_100_km'])
print(f"Direct Correlation: {direct_correlation:.4f}")

Direct Correlation: 0.9181
```

This technical validation mathematically proves the project's main business rule: fuel consumption translates directly into emissions, which is why the efficiency_multiplier is so crucial to the reward logic.

- **Engine_Size_L and Cylinders:** Act as the secondary fundamental baseline for the prediction tree. Larger combustion chambers raise the minimum possible emissions floor, regardless of driving efficiency.
- **Fuel_Type & Vehicle_type:** These categorical features act as important branching conditions in the Random Forest, adjusting penalty/baseline metrics based on inherent combustion properties and vehicle weight class (e.g., SUVs vs. Compacts).